Worksheet — 1.2 Software & Software Development — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

1. Key-term match: 1 → C; 2 → E; 3 → A; 4 → F; 5 → D; 6 → B.

2. Stages of compilation: - Stage 1 — **Lexical** analysis: code → **tokens**; remove whitespace/comments; build **symbol** table. - Stage 2 — **Syntax** analysis (also called **parsing**): tokens checked vs **grammar**; **parse** tree built; **syntax** errors reported here. - Stage 3 — Code **generation**: produces **object** / machine code from the parse tree + symbol table. - Stage 4 — Code **optimisation**: refines code to run **faster** / use less **memory**, same result.

3. Translators:

	Compiler	**Interpreter**
When	All at once, before the program runs	Line by line, at run time
Output	Standalone executable (.exe)	None
Program speed	Fast (already machine code)	Slower (translated each run)
Source needed to run?	No (executable only)	Yes, plus the interpreter
Errors	All reported at the end	Stops at the first error found
Best for	Distributing finished software	Developing/debugging, scripting

Assembler: translates **assembly language** into machine code, at roughly **one mnemonic : one machine instruction** (one-to-one).

4. Methodology match: a) **Spiral**; b) **Waterfall**; c) **RAD**; d) **Extreme Programming (XP)**; e) **Agile**.

5. Addressing modes (worked): For `LDA 20` with [20]=35, [35]=99, [99]=12: - a) **Immediate** — operand *is* the value → loads **20**. - b) **Direct** — operand is the address holding the value → go to address 20, find 35 → loads **35**. - c) **Indirect** — operand is the address of a location holding the *address* of the value → address 20 holds 35; address 35 holds 99 → loads **99**. - d) **Indexed** — operand = base address + contents of the **index register**; used to step through **arrays**.

6. Scheduling & interrupts: - a) **Round robin** — it **is** pre-emptive (the time slice forces a switch). - b) Any one: requires each job's total runtime to be known/estimated in advance; **long jobs can starve** if short jobs keep arriving; not pre-emptive once a job starts. - c) **Pre-emptive** = a running process can be stopped (interrupted) and the CPU reallocated to another process before the first finishes. - d) Order: CPU checks interrupt register (1) → finish current instruction if higher priority (2) → push registers onto stack (3) → load address of correct ISR (4) → run ISR (5) → pop registers off stack and resume (6).

Step	Order
Load the address of the correct ISR	4
Pop the registers off the stack and resume the original task	6
CPU checks the interrupt register at the end of the F-D-E cycle	1
Run the ISR	5
Push the registers (e.g. PC, ACC) onto the stack	3
Finish the current instruction (if the interrupt is higher priority)	2

- e) A stack is **LIFO**, so when a higher-priority interrupt occurs while another ISR is running (**nested interrupts**), the registers are restored in the correct reverse order and every task resumes exactly where it left off.

7. OOP match: instance → **Object**; template/blueprint → **Class**; variable/data → **Attribute**; subroutine/behaviour → **Method**; bundling + data hiding → **Encapsulation**; subclass acquiring superclass features → **Inheritance**; same method name, different behaviour → **Polymorphism**.

8. Challenge: - (i) **Compiler.** Reasons (any two): it produces a **standalone executable** so customers do **not** receive the source code; the program runs **fast** because it is already machine code; the user does **not** need the source or an interpreter installed to run it. - (ii) Virtual memory uses **secondary storage (a swap/page file)** *as if* it were RAM — the physical RAM is unchanged, and disk access is much **slower** than real RAM. Excessive swapping of pages between RAM and disk causes **disk thrashing**, where the machine spends most of its time paging and grinds to a halt.